SOFTABLITZ 2026

TEAM: JARVIS

Team ID

261

Participation: Solo participant

TEAM INFO

Member 1: Tarun Pratap Singh (20253301, CSE, tarunsingh6040)

Member 2: N/A — solo participant

Member 3: N/A — solo participant

DESCRIPTION OF THE PROBLEM

Eye of Agamotto is a desktop application for observing system resource usage through a continuously updated interface. It reports CPU utilization, memory consumption, disk usage, and battery status where supported by the operating system. Users can inspect running processes and identify resource-intensive applications. The design prioritizes responsive monitoring, clear presentation of metrics, and safe handling of unavailable data and permission errors.

App name: Eye of Agamotto

PROCESS FLOW OF THE APP

1. Initialize the desktop interface and detect which system metrics are available.

2. Collect CPU, memory, disk, and battery metrics at a configurable interval.

3. Validate each sample and pass a normalized snapshot to the dashboard.

4. Update metric cards and time-series graphs without blocking user interaction.

5. Retrieve process details on demand and allow sorting by CPU or memory usage; handle inaccessible or terminated processes.

6. Compare samples against user-defined thresholds and display alerts when limits are crossed.

7. Stop timers and background workers cleanly when the user exits.

Features announcement

Core: Live CPU, RAM, and disk usage; battery percentage and charging status when available.

Dashboard cards and live graphs with a configurable refresh interval.

Process explorer with CPU and memory usage, sorting, and graceful handling of permission errors or processes that exit.

Configurable resource thresholds and desktop notifications.

Stretch: Compact always-on-top mode, process termination with confirmation, and a packaged desktop build.

Platform-specific checks show unsupported metrics as unavailable rather than presenting fabricated values.

External libraries/APIs

Planned libraries: psutil for cross-platform system metrics and process information; PySide6 (Qt for Python) for the desktop interface, widgets, timers, and safe UI updates; and pyqtgraph for frequently updated time-series graphs.

Selection rationale: psutil avoids separate low-level OS calls for each supported metric; PySide6 offers more flexibility for a rich dashboard than Tkinter, though packaging is more involved; pyqtgraph is focused on interactive real-time plotting, while Matplotlib is often better for static or less frequently refreshed charts.

Library selection and engineering considerations:

1. psutil collects CPU, memory, disk, battery (where supported), and process information through one cross-platform API.
2. Direct operating-system APIs were avoided for the MVP because they require separate implementations for different platforms.
3. PySide6 offers richer widgets and integration options for a monitoring dashboard than a minimal toolkit such as Tkinter.
4. Qt signals and timers support UI updates from background collection without directly modifying widgets from a worker thread.
5. pyqtgraph is intended for interactive, frequently updated graphs; Matplotlib is often better for static or less frequently refreshed plots.
6. A local-first desktop app avoids a remote backend and network dependency for core monitoring, reducing unnecessary telemetry exposure.
7. psutil values may be unavailable on some systems or restricted for some processes; optional values and access errors must be handled.
8. A fixed-size history buffer bounds memory use, unlike indefinite retention of every graph sample.
9. A configurable refresh interval and reasonable process-query cadence balance responsiveness with resource consumption.
10. Package versions, OS compatibility, and desktop packaging will be checked during implementation before release.

Core Engineering Architecture

The **Design Approach** separates metric collection, validation, bounded history, and UI rendering to keep the monitor responsive.

Infrastructure layout and data flow: The app is local-first: operating-system APIs feed psutil's collector, which validates and normalizes a metrics snapshot. A bounded in-memory history buffer supplies time-series graphs, while the dashboard and process view display live readings. No backend server or database is needed for core monitoring, avoiding network dependence and unnecessary exposure of system telemetry.

Resource use and responsiveness: Metrics are sampled at a controlled interval. Collection runs outside the GUI thread; background results pass to the interface using Qt signals or a timer-driven worker design. Graph history retains a fixed time window, and process details are queried at a reasonable cadence.

Naive approach and failure mode: Collecting metrics synchronously in GUI event handlers can freeze the interface. Keeping every graph sample indefinitely can grow memory use, while assuming all process fields are accessible can cause failures on restricted or unsupported data.

- **Naive approach:** Synchronous collection in GUI event handlers can freeze the interface, while unbounded graph history increases memory use.
- **Engineering approach:** Separate collection from presentation, validate optional values, catch expected access and process errors, bound graph history, and make refresh intervals configurable. Require confirmation before process termination and avoid targeting protected or critical system processes by default.
- **Why it helps:** These safeguards improve responsiveness, limit resource use, and reduce the impact of a single unavailable metric.

Class Diagram

MetricsCollector collects system metrics into a MetricsSnapshot; DashboardWindow renders it. ProcessService lists running processes, and HistoryBuffer stores a fixed time window of samples.

